

AI Header Resolution for Bulk Policy Upload

Automatically mapping partner CSV headers to our canonical policy schema, so bulk upload scales from tens of partners to a thousand without scaling the manual effort behind it.

24

PARTNER FORMATS ANALYSED

86%

HEADER REUSE MEASURED

~6

TEMPLATE FAMILIES

— Contents

1. Executive summary	The ask in one page
2. The problem and where it sits	Current flow, break point
3. Evidence from partner data	What 24 formats tell us
4. Findings that shape the design	Five things that change the build
5. Proposed architecture	Two-level mapping
6. The resolution pipeline	Five layers, cheapest first
7. When we call the LLM	Invocation policy and cost
8. The learning dictionary	How the system gets cheaper
9. Gaps in our master schema	Decisions only the business can make
10. Delivery plan	Phases and sequencing
11. Risks and controls	What can go wrong
12. Decisions required	What we need from this room
A. Appendix	Families, schema properties, caveats

How to read this document

Sections 1–2 frame the problem. Sections 3–4 present what we learned from analysing real partner files — this is the evidence base, and several findings overturn assumptions we started with. Sections 5–8 are the proposed solution. Sections 9–12 are what we need decided.

If you read only one thing

Our partners do not send 24 different formats. They send roughly **six**, and 86% of their column headers are shared between them. That single fact is what makes a thousand partners affordable, and it means the first useful version of this system needs **no AI at all**.

1 Executive summary

Partners upload policy CSVs whose column headers do not match ours. Today a person fixes them by hand. That works at ten partners and fails at a thousand. We propose a header resolution service that maps partner columns to our canonical fields automatically, escalating only what genuinely needs judgment — and that gets cheaper as it learns.

The goal, stated as a number

Target outcome

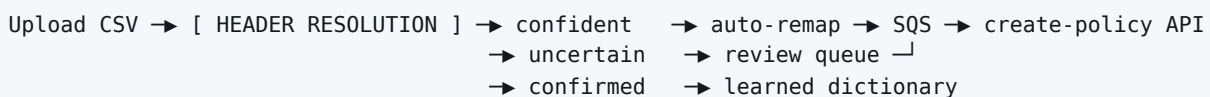
Partner count grows from ~10 to 1000+. **Person-hours spent fixing headers stays flat. Wrong mappings stay at zero.**

What we found in the data

Finding	What it means for us
86% header reuse	A learned dictionary resolves most columns of a new partner before any model is consulted.
~6 template families , not 24 formats	Onboarding partner 1000 becomes "which family, then what is different" rather than mapping 60 columns from scratch.
Three partners are byte-identical pairs	Same upstream vendor system serving several lenders. Mapping one maps all.
Some partners repeat unmarked person columns	The most dangerous case we found. Detailed in Section 4.
Several common partner fields have no home in our schema	Needs a business decision, not an AI solution. Section 9.

What we propose

A pre-flight header resolution service sitting between upload and the existing SQS queue. **The current pipeline is untouched.** Columns are resolved by a tiered pipeline that tries the cheapest method first and only reaches an LLM for headers that genuinely need reasoning. Every human confirmation is written back, so the free layers cover more traffic over time.



Cost shape

LLM cost scales with formats, not with files or rows

A partner is mapped once at onboarding — roughly **one model call**. Their template is then pinned and subsequent uploads cost **nothing**. Approximately **1,000 model calls in total, ever**, versus hundreds of millions if we called a model per row.

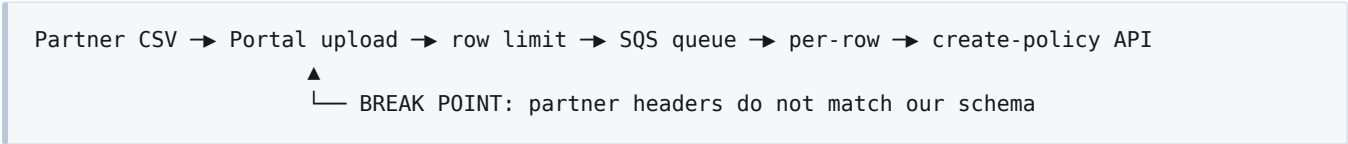
What we are asking for

1. Approval of the phased approach in Section 10
2. Answers to the seven decisions in Section 12 — two of which block real work
3. Access to historical partner files and the record of how they were manually fixed
4. An owner for the review queue

2 The problem and where it sits

Today's flow works — until the headers are wrong

The bulk upload portal accepts a partner CSV, applies a configurable row limit, pushes rows to SQS, and each row calls the create-policy API. This pipeline is reliable and is not the subject of this proposal.



When a partner's headers do not match our canonical schema, the file cannot proceed. Today a person resolves that by hand. The effort is roughly linear in the number of partners, which is precisely the quantity we intend to grow by two orders of magnitude.

Scope of this proposal

In scope

- Deciding which canonical field each uploaded column corresponds to
- Escalating what cannot be decided confidently
- Learning from every human correction
- An audit trail of every mapping decision

Out of scope

- Row-value validation
- The SQS pipeline
- The create-policy API
- Policy business rules

A question worth asking before we build anything

If a partner template exists and partners ignore it, upstream enforcement may be higher leverage than downstream mapping. The evidence suggests we need both: **only 4 of 24 partners adopted our template**, and the rest are sending their core banking system's native export. They largely cannot conform, so mapping downstream is the pragmatic path.

What we still do not know

The following are unanswered and are the subject of a requested working session with operations:

Question	Why it matters
% of files that currently need a header fix	Baseline for the target
Person-hours per week spent on this today	Sizes the benefit
Who fixes them, and how they know the right mapping	Becomes our labelled training data
Whether a fix is remembered for the partner's next file	Tells us if the problem is repeated or one-off
What a wrong mapping costs downstream	Sets our auto-apply confidence threshold

3 Evidence from partner data

We analysed 24 partner header sets — 19 loan, 5 health, 0 travel. 1,113 header instances, 346 distinct after normalization. The following are measurements, not estimates.

3.1 The decisive number: 86% reuse

86% of header instances use headers that appear in two or more partners

191 of 346 distinct headers (55%) are shared across partners. The long tail is real, but small. **A dictionary seeded from these 24 partners will resolve most columns of partner 25 for free.**

The most widely shared headers, and how many of the 24 partners send each:

```
dob 22      ·   loan amount 19      ·   mobile no 19      ·   gender 17      ·   no of lives 17
loan tenure 16      ·   cover type 16      ·   pin code 16      ·   relation 16      ·   transaction id 15
nominee dob 14      ·   application date 13      ·   tenure 13      ·   sum insured 13      ·   pep 13
```

3.2 Partners cluster into roughly six families

Measuring overlap between every pair of partners (Jaccard similarity on normalized headers) shows clear clustering rather than 24 unique formats:

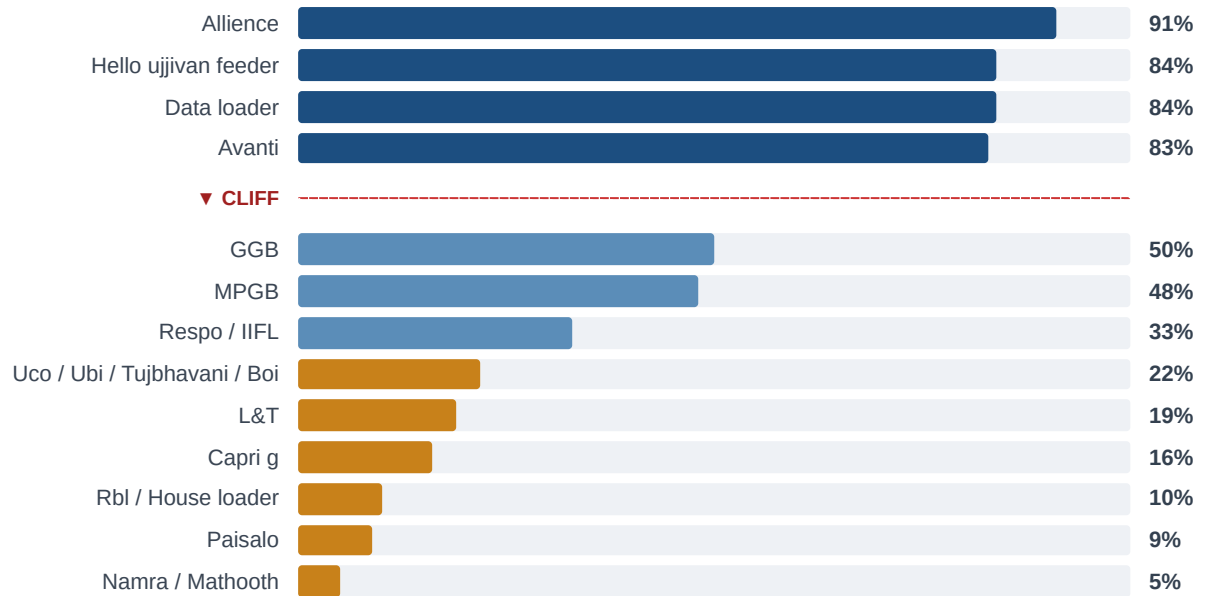
Family	Members	Similarity
Vendor-identical	Rbl ≡ House loader · Namra ≡ Mathooth · Hello ujjivan feeder ≡ Data loader	1.00
Master-aligned	Allience, Avanti, Hello ujjivan feeder, Data loader	0.64–0.69
Bare-repeat	MPGB, GGB	0.90
Proposer/Applicant	Ubi, Boi, Hospital partner, Capri g, Uss, Uco, Agri feeder, Hello ujjivan standalone	0.55–0.78
camelCase health	Namra, Mathooth	1.00
Own-format singletons	L&T, Paisalo, Respo, IIFL, Tujbhavani, Sambhavati	—

Why this is the most important commercial finding

Three pairs are **exact duplicates** — the same upstream vendor system serving multiple lenders. Partner 1000 is overwhelmingly likely to be a variant of a family that already exists, not a new format. A per-partner mapping table scales with *partners*; family templates scale with *formats* — and formats grow far more slowly.

3.3 Coverage against our schema is bimodal, not a gradient

Percentage of our 58 canonical fields that each partner sends verbatim:



There are two populations here, not a spectrum: partners who **adopted our template** (83–91%) and partners who send **their own core system's export** (5–33%). They need different treatment — the first needs drift detection, the second needs real mapping. This also tells us where to start the rollout.

4 Findings that shape the design

Five findings from the partner data overturn assumptions we began with. Each changes what we build.

F1 Column position is load-bearing — and this is the dangerous one

MPGB and GGB repeat **bare, unmarked** person columns three times in a single row. Real column positions from the MPGB file:

15 Customer ID	16 Title	17 First Name	18 Last Name	19 DOB	← PROPOSER
...					
33 Customer ID	35 Title	36 First Name	37 Last Name	38 DOB	← INSURED
...					
	41 Title	42 First Name	43 Last Name	45 DOB	← NOMINEE

The string "First Name" maps to three different fields in one file

Column 17 → First Name , column 36 → First Name_Plan , column 42 → Nominee First Name . The three headers are **byte-identical**. No LLM, no embedding and no fuzzy match can separate them — only ordinal position can.

A system keyed purely on header text maps all three to the proposer, and **the wrong person is insured**. Identically for DOB (19/38/45) and Title (16/35/41).

Design consequence: the unit of resolution must be (position, header) , never header alone.

F2 One column does not always mean one field

Nominee Name arrives as a **single column in 9 of 24 partners**, but our schema has Nominee First Name and Nominee Last Name . The same applies to name (Namra, Mathooth), custname (Rbl) and NOMINEE_NAME (L&T).

Design consequence: splitting affects roughly a third of the partner base. Transforms are core functionality, not an edge case.

F3 Health is a different schema, not a variant

Rbl and House loader carry **repeating insured groups**:

```
insured_adult2, gender_2a, relationship_2a, dob_2a, age_2a
insured_child1, gender_c1, relationship_c1, dob_c1, age_c1
insured_child2, gender_c2, relationship_c2, dob_c2, age_c2
```

Our schema models **exactly one** insured. A multi-life health row cannot be represented at all. This is a schema gap, not a mapping problem.

Design consequence: separate canonical schemas per product — loan_v2 , health_v1 , travel_v1 .

F4 Some synonyms have zero string overlap

Partner header	Our field	Fuzzy match score
APPOINTEE_NAME (L&T)	Guardian FirstName	~0 — no shared characters
SUM_ASSURED / SumAssured	Sum Insured	low
GWP (Paisalo)	Amount	0
custname (Rbl)	First Name	low
constructType (Namra)	Cover Type	low

Appointee is the correct insurance term for the guardian of a minor nominee. String similarity will never find it; semantic embeddings will. **This is precisely where an embedding layer earns its cost** — these cases are semantically obvious and lexically invisible.

F5 Typos are systematic, and free to fix

Proposal no · Aplication Date · Intetemediry Code · Intemediary Code · Nomini Name
Transation Date · Disbursment Date · Loan disbusrement date · Convalescene · Booking Scop-Secondry

Pure fuzzy-matching territory — no model needed. Note that `Member No.` is sent by 6 partners but our field is `Member Number`: nobody sends it verbatim, yet fuzzy matching recovers it instantly.

F6 Header junk is real and must be tolerated

- Entire declaration paragraphs (~80 words) used as column headers, in **7 partners**
- Leaked BI tool references: `'GTLLead'[FirstameComplete]`
- Embedded business rules: `Policy Tenure ( Cannot be less than Loan tenure subject to maximum of 5 years)`
- Merged concepts: `AadharAddressLine1`, `Convalescence Benefit/EMI Value`
- Duplicate columns within one file (Mathooth sends `memberId` twice)

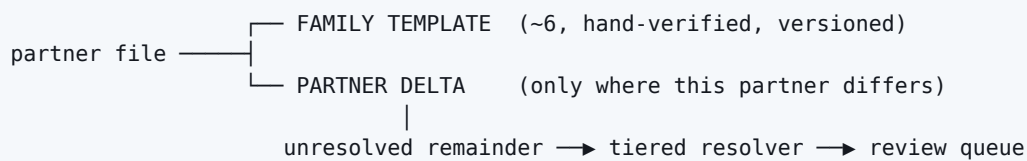
5 Proposed architecture

5.1 Where it sits

A standalone service called by the portal at upload time, **before enqueue**. The existing SQS pipeline and create-policy API are untouched. Running it as a separate service (rather than embedding it in the portal) lets claims, renewals and commission uploads share one dictionary later, rather than each building an isolated one.

5.2 Two-level mapping — the main design decision

We do not resolve every file from scratch. We resolve against a **family template**, then a **partner delta**.



Why this makes 1000+ partners tractable

Onboarding becomes: classify into a family (cheap and high-signal, given 0.55–1.00 intra-family similarity), inherit its verified template, then review only the delta. **Given 86% reuse, the delta is small.**

5.3 Per-product schemas

Driven by finding F3. The alias dictionary is scoped by `(tenant, schema_id)` so that domains cannot poison each other — `amount` means premium in policy and claim amount in claims, and one flat dictionary would let one corrupt the other.

5.4 Everything is auditable

Every mapping decision is logged: source header, chosen field, confidence, which layer resolved it, who confirmed it, and when. This is what allows a bad mapping discovered six months later to be traced to **exactly which files used it** — without which the affected policies cannot be remediated. For regulated data we expect this to be a requirement, not a nicety.

6 The resolution pipeline

Cheap deterministic methods resolve the easy majority. The model only sees headers that genuinely need judgment.

Layer	Method	Expected load	Cost
0	Normalize — case, separators, camelCase, abbreviations, truncate parentheses	all columns	FREE
1	Family template + learned alias dictionary	the bulk	FREE
2	Fuzzy string matching	typo tail (F5)	FREE
3	Semantic embeddings	synonym tail (F4)	cheap
4	LLM reasoning	entity assignment, novel structures	PER CALL

The LLM's real job is narrower than "map headers"

It is **entity assignment and structural decisions** — which of four people a column describes, and whether a column needs splitting. That is precisely where a wrong answer creates a wrong policy, and precisely what cheap methods cannot do.

Why we send the whole schema, not a shortlist

An earlier prototype shortlisted candidate fields by string similarity before asking the model. That is self-defeating: `Premium Amt` has no textual overlap with `Amount`, so ranking pruned the correct answer out of the list and forced an abstention. All 58 fields go into the system prompt — about 900 tokens, identical on every call, so it sits in the cacheable prefix and costs almost nothing.

Sample values do the narrowing instead

Of the 47 confusable field pairs in our schema, **22 are separable by looking at one value** and 25 are not:

Type-separable (22) — values solve these

Transaction Id vs Transaction Date
Loan A/c No vs Loan Amount
Occupation vs Occupation_Class

Type-identical (25) — values useless

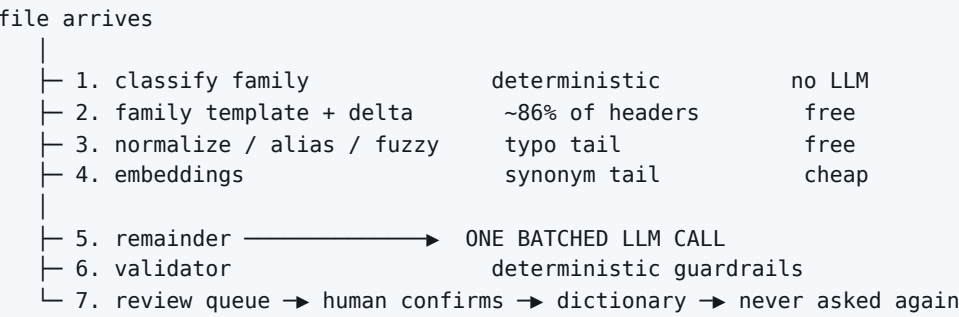
First Name vs First Name_Plan
DOB vs DOB_Plan
Address Line 1 vs Address Line 2

Our schema has 8 date fields that are mutually indistinguishable by value — only header semantics and column position separate them. A partner sending `Date1`, `Date2`, `Date3` cannot be resolved by any technique and must go to a human.

7 When we call the LLM

7.1 Invocation policy

Per unseen header, once ever — not per row, not per file



7.2 What does *not* trigger a call

- A file from a partner whose template is pinned and whose headers have not drifted — **zero calls**
- Any header already in the dictionary for that product
- Repeat rows — resolution is per *file header set*, never per row

7.3 The cost argument

Approach	Calls	Notes
Proposed — once per new format	~1,000 total, ever	~1 call per partner at onboarding; nothing thereafter
Naive — per row per column	~600,000,000	1000 partners × 10k rows × 60 columns

Call volume also **decays over time**: every confirmed mapping enters the dictionary, so the same header is never sent to a model twice.

7.4 Why we batch the whole file into one call

This is a correctness requirement, not an optimisation. Finding F1 showed that MPGB's three identical `First Name` columns are separable only by their position in the ordered list. **Sending headers one at a time destroys the only signal capable of resolving them.** The model must see the full ordered header list, with indices.

7.5 The validation gate — where the safety actually lives

LLM output is a **proposal**, never applied directly. Every proposal passes deterministic checks, none of which involve AI:

Check	Action on failure
Target field exists in our schema	Reject — hallucination
Sample values match the target's data type	Reject — e.g. dates proposed for a decimal field
No two columns claim the same field	Conflict — both to review
Entity consistent with the surrounding block	Flag for review
All 9 required fields resolved	File gated
Confidence above threshold	Below — suggestion only, stays in review

Abstention is a first-class answer

The model is explicitly permitted and encouraged to answer "I don't know." An extra row in the review queue costs a few seconds. A wrong auto-applied mapping silently corrupts a policy record.

7.6 Is this an "AI agent"?

Mostly no — and that is the right answer. For the large majority of cases, a single well-contexted structured call plus the validator outperforms a multi-step agent: lower latency, lower cost, fewer failure modes, and a far easier audit story for a regulated decision.

An agentic loop earns its place in one situation: **the header carries no information and the answer is in the data** — `col_17` , `Date1/Date2/Date3` . There we give the model tools to inspect the column:

Tool	Resolves
<code>get_column_values(index, n)</code>	More samples than the default window
<code>check_format(index)</code>	PAN regex, mobile, ISO date, currency
<code>distinct_count(index)</code>	Identifier vs categorical
<code>compare_columns(i, j)</code>	Duplicate or correlated columns

Recommendation: build the single call first. Add tools only when the review queue demonstrates cases a single call cannot solve. Let the queue decide what gets built, not what is technically interesting.

7.7 Failure behaviour

The system must be fully functional with the LLM disabled — **degraded coverage, never degraded correctness**. A model timeout leaves the header unresolved and the file proceeds to review. A provider outage affects only the long tail; the deterministic layers continue unaffected.

8 The learning dictionary

This is the mechanism that makes the system get cheaper as partners grow, rather than more expensive.

The intuitive design — store each master header as a key and all similar partner headers as its values — is the right core idea. Four properties of the real partner data require it to be refined.

Simple form	Refinement	Forced by
Key on our master field	Key on the partner alias; keep master → aliases as a UI view	Lookup direction at runtime
Key = header string	Key = (tenant, schema, scope, alias, occurrence_index)	F1 — "First Name" has 3 targets in one file
Value = one field name	Value = {targets[], transform, entity}	F2 — Nominee Name → 2 fields
One flat table	Three scopes: partner → family → global	"Tenure" means different things per partner
Write on validation	provisional → active on confirmation; negative on rejection	Stops the loop amplifying its own errors

8.1 Three scopes, with promotion

Scope	Holds	Example
global	Universally safe aliases	Propsoal no → Proposal
family	One of the ~6 template families	Proposer/Applicant FIRST Name → First Name
partner	Partner overrides and positional entries	MPGB First Name occurrence 1 → First Name_Plan

Lookup runs **partner** → **family** → **global**, most specific wins. New mappings are always written at partner scope, where they are cheap to correct. They are *promoted* to family or global only after appearing in several partners with zero contradictions.

The single most important rule in this design

Positional entries never promote. MPGB's "First Name at occurrence 1 means the insured" is true only for that column layout. Promote it to a global rule and it corrupts every other partner in the system. Every other mistake in this model is recoverable; this one is not.

8.2 Lifecycle

Status	Meaning	Behaviour
provisional	Applied without human confirmation	Used, but re-verified; promotes after N clean files
active	Human-confirmed	Auto-applies
negative	Human rejected this mapping	Never suggested again
retired	Superseded or schema changed	Ignored, kept for audit

Negative entries matter more than people expect. Without them the model re-proposes the same rejected mapping every month, and operations re-rejects it every month.

A subtle failure we are designing against

If every auto-applied mapping is written back as fact, a confident but wrong model output becomes permanent and self-reinforcing — the dictionary teaches itself the mistake. The `provisional` state exists specifically to prevent this.

8.3 A warning on normalization

Normalization must not be too aggressive. `Loan Tenure` and `Policy Tenure` must **not** collapse to `tenure` — the qualifier carries the meaning, and 16 and 9 partners respectively send them as distinct columns. Stripping leading qualifiers is where this design would quietly break.

9 Gaps in our master schema

No AI can map a column to a field that does not exist

These are business decisions, not engineering problems, and they block real mapping work regardless of how good the AI is.

9.1 Fields many partners send that we have nowhere to put

Partner header	Partners	Issue
Gender	17 of 24	No Gender field exists in our schema at all
Cover Type (Individual/Floater)	16	Referenced inside our own Sum Insured field description, but is not itself a field
Policy Tenure	9	We have one Tenure ; partners send Loan Tenure and Policy Tenure separately
Actual Premium / Transaction Amount	8 / 7	We have one Amount — which of the two is it?
Good Health Declaration	7	Underwriting-relevant
PED block (flag / remarks / month)	6	Pre-existing disease
State	6	We have City and Pin Code but no State
Proposal No	6	—

Gender is absent from a life and health master schema

17 of 24 partners send it. This is worth escalating on its own merits, independently of this project.

9.2 The two ambiguities that most concern us

Tenure is our sharpest trap. We have a single field; partners send two distinct columns. Mapping it wrong silently changes policy duration — a defect that is easy to create and hard to detect.

Amount is a *required* field whose meaning is currently ambiguous against three distinct partner columns (Actual Premium , Transaction Amount , Premium Amount). We need a definitive answer before we can map it safely.

9.3 Our nine required fields

A file cannot create a policy without these. They are the real gate — **not** whether every column mapped:

Member Number · Application Date · Loan A/c No · Customer ID · First Name
Last Name · DOB · Amount · Sum Insured

This reframes partial-file handling usefully: an unmapped GST Registration Date should never block an upload. We recommend gating on the nine required fields only.

10 Delivery plan

Phase	Content	Timing	Exit criteria
0	Frame the problem — operations walkthrough of one real file	Week 1	Problem statement and agreed target number
1	Ground truth — historical files and how they were fixed	Weeks 1–2	≥300 labelled header → field pairs PARTLY DONE
2	Lock the decisions in Section 12	Week 3	Dated decision log with owners
3	Architecture — service, API contract, data model	Week 4	Design doc signed off
4	Build in thin slices (below)	Weeks 5–10	Each slice green against the eval set
5	Calibrate thresholds against real data	Week 11	Zero wrong auto-applies, coverage maximised
6	Shadow mode — produce mappings, apply nothing	Weeks 12–15	Agreement rate above target
7	Staged rollout, per partner, with kill switches	Week 16+	—
8	Extend to claims, renewals, commissions	Later	—

10.1 Build slices

Slice	Content	Ships alone?
S1	Normalize + family templates + alias dictionary + audit log	YES — NO AI
S2	Review UI + confirmation loop feeding the dictionary	yes
S3	Fuzzy matching + position-aware entity resolution	yes
S4	Semantic embeddings for the synonym tail	yes
S5	LLM for entity assignment and novel structures	yes
S6	Transforms (splitting a name column into two fields)	yes

10.2 Two phases we recommend not compressing

Phase 1 is a gate, not a task. Collecting historical files and how they were fixed is unglamorous and is not AI work, but it is the only thing that makes our confidence thresholds defensible rather than invented.

Shadow mode is the only honest validation. Running in production while applying nothing, and comparing every suggestion against what the human actually did, is the only way to know the system is safe before it touches a policy.

10.3 Where to start the rollout

Begin with the **master-aligned family** — Allience, Avanti, Data loader, Hello ujjiyan feeder. At 83–91% coverage the risk is near zero and the volume is immediate.

If the timeline compresses: ship S1 + S2 only, with no AI

Normalization, the six family templates, an alias dictionary, and a review loop where every confirmation is remembered. With 86% header reuse this alone should remove most of the manual effort — no model cost, no latency in the upload path, and no data-residency question to answer.

Add embeddings when the synonym tail dominates the review queue, and the LLM when entity ambiguity does.

Sequence the AI by what the review queue is actually full of, not by what is technically interesting.

11 Risks and controls

Risk	Control
Wrong entity → wrong person insured The F1 failure mode	Position-aware resolution; unmarked repeated blocks are <i>never</i> auto-applied; flag rather than guess when entity is uncertain
Schema gaps silently drop partner data	Decisions in Section 9 before build; any column that is populated but unmapped must be reported, never silently discarded
The dictionary learns its own mistakes	<code>provisional</code> status; promotion requires clean files or human confirmation; positional entries never promote
A wrong mapping is discovered months later	Append-only decision log identifies exactly which files used the bad alias, so affected policies can be remediated
Health multi-life cannot be represented	Separate <code>health_v1</code> schema; do not force into the loan schema
No usable ground truth exists	Phase 1 is a gate; thresholds cannot be justified without it
Policy-holder data sent to an external model	Headers and redacted samples only; or in-region hosting. Requires a compliance answer before Phase 4
Nobody owns the review queue	Forced to an answer in Week 3. If unowned, "flag for human" is a dead end and the design must change
LLM provider outage	System is fully functional with the model disabled — degraded coverage, never degraded correctness

The failure we are most concerned about

A confidently wrong mapping is worse than no mapping

A file that stops and asks a human costs minutes. A file that proceeds with the nominee's name in the insured's field creates a policy that is wrong in a way nobody notices until a claim is made. Every threshold in this design is set to prefer the first outcome.

12 Decisions required

Each needs a named owner and a written answer. Two are blocking.

#	Decision	Why it matters
D1	When does mapping run — per upload, once at onboarding, or hybrid?	Determines whether we build a runtime service or a configuration tool. <i>We recommend hybrid.</i>
D2	Who resolves what the system cannot?	If no one owns a review queue, "flag for human" is a dead end
D3	What does a wrong mapping cost — detectable, reversible, regulated?	Sets the auto-apply threshold, which is currently a guess
D4	What happens to a partially resolvable file?	<i>We recommend gating on the 9 required fields only</i>
D5	Product scope — loan, health, travel?	Each needs its own canonical schema
D6	Schema gaps — do we add Gender, Cover Type, Policy Tenure, State and the PED block, or explicitly drop them?	BLOCKING No AI resolves a column with no target
D7	What does our Amount field actually mean?	BLOCKING A required field, ambiguous against three partner columns

What we need from this room

Stakeholder	Ask
Operations	One working session walking a real partner file end to end, including a case that needed manual fixing. Access to historical files and any record of past fixes.
Product / business	D1, D2, D4, D5, and the schema-gap decisions D6 and D7. A definition of success as a number.
Engineering	Confirmation the service can sit before enqueue. Approved model and embedding providers, region, and hosting.
Risk / compliance	D3 — the cost of a wrong mapping. Whether automated mapping of policy data is acceptable and under what controls. Constraints on sending data to a model.

Recommended immediate next step

One working session with operations, this week

Walk a single real partner file from arrival through to policy creation, including a case that needed manual header fixing. Bring the file, and whoever fixed it. That one walkthrough answers most of Section 2 and several of the decisions above — and those historical fixes become the labelled data set we need to set thresholds honestly.

A Appendix

A.1 Our master schema, in numbers

Property	Value
Canonical fields	58
Required fields	9
Person-attribute fields	18, across 4 entities (proposer 5, insured 5, nominee 5, guardian 3)
Confusable field pairs	47 — of which 12 span different people
Separable by sample values	22 pairs
Not separable by values	25 pairs — need header semantics or position
Date fields	8, mutually indistinguishable by value
Small enough to skip a vector database?	Yes — 58 fields fits comfortably in a prompt

A.2 The four people sharing one row

Our schema packs **proposer, insured, nominee and guardian** into a single row. This produces a *systematic* rather than random error: the unqualified variant is always the shortest string, so naive matching scores a bare `First Name` highest against the **proposer** every single time — even in a file that means the insured.

Resolution is therefore two-stage: decide **whose** column this is, then **which** attribute. Three signals, in descending reliability:

1. **Explicit marker** — `Nominee First Name`, `First Name_Plan` → certain
2. **Sibling exclusion** — a file containing both `First Name` and `Nominee First Name` proves the bare one is not the nominee → strong
3. **Block adjacency** — person blocks arrive contiguously, so a decided neighbour is real evidence

Where none apply, the entity is a *guess* and the mapping is flagged rather than applied.

A.3 Partner formats analysed

Partner	Product	Naming convention
L&T	Loan	UPPER_SNAKE
MPGB, GGB	Loan	Title Case — unmarked repeated blocks
Tujbhavani	Loan	Mixed
Sambhavati	Loan	Title Case — metadata block only
Respo	Loan	snake_case
Paisalo	Loan	PascalCase / concatenated
Namra, Mathooth	Health	camelCase — identical to each other
IIFL	Loan	Mixed, with leaked BI references
Capri g, Uss, Uco, Ubi, Boi, Agri feeder	Loan	Title Case — "Proposer/Applicant" family
Avanti, Allience	Loan	Title Case — master-aligned
Rbl, House loader	Health	snake_case — multi-life, identical to each other
Hospital partner	Health	Title Case
Hello ujjivan standalone / feeder	Loan	Title Case
Data loader	Loan	Title Case — master-aligned

A.4 Caveats on this analysis

Two limits on the evidence presented here

1. The partner headers were reconstructed from a PDF. The source document lost header boundaries — word gaps and column gaps were both rendered as the same spacing — so the header lists were rebuilt using domain judgment. The structural conclusions are robust: 86% reuse would have to be wildly wrong to change them. The precise figures are not eval-grade. **We need the original CSV or Excel files before using this to set thresholds.**

2. Travel was named as a product category but no travel samples were supplied. The analysis covers 19 loan and 5 health formats. No design claim is made for travel.

A.5 Glossary

Term	Meaning
Canonical field	One of the 58 columns in our master policy schema
Alias	A partner's name for one of our canonical fields
Family template	A verified mapping shared by a cluster of similarly-formatted partners
Entity	Which of the four people a column describes — proposer, insured, nominee, guardian
Auto-apply threshold	Confidence above which a mapping is applied without human review
Abstention	The system answering "I don't know" rather than guessing
Shadow mode	Running in production producing mappings but applying none, to measure accuracy safely